MiK FRONTEND BUILD DIRECTIVE

Project: Arvin Dispensary OS — Operator Console
Target: `/home/ubuntu/arvin_dispensary_os/frontend`
Consumes: the persisted JSON artifacts already produced by the backend modules (Competitor Radar, Margin & Dead-Stock Scanner, Forward Intelligence).
Status of prerequisite: Step 1 verified — `data/forward-intel/weekly-dossier-1787270627557.json` exists (Exit Code 0).

---

0. MISSION

Scaffold a **read-only operator dashboard** that renders the data layer already written to `data/**`. The frontend is a viewer, not an actor. It reads persisted JSON, never Dutchie, never AWS, never a write path. It surfaces exactly what the backend chose to surface — including TIER_2/TIER_3 alert state — and never re-derives or "fixes" backend numbers.

This directive is executable as-is. Every acceptance gate at the bottom must pass before commit.

---

1. HARD RULES (INHERITED FROM BACKEND CANON — NON-NEGOTIABLE)

1. **Read-only.** The frontend performs NO writes, NO Dutchie calls, NO AWS calls, NO outbound sends (email/webhook/SMS). It only reads local JSON artifacts from `data/**`.
2. **No secrets in the frontend.** No API keys, no `DUTCHIE_*`, no AWS creds may appear in any file under `/frontend`, in any bundle, or in any env var exposed to the browser. If a loader needs the filesystem, it runs **server-side only**.
3. **Never fabricate data.** If an artifact is missing, empty, or a section is zeroed (e.g. `totalVendors: 0` because the live pull 403'd), the UI must render an explicit **"No data / upstream unavailable"** empty state — never placeholder/mock numbers, never interpolated values.
4. **Surface alert tiers faithfully.** TIER_2 = visible degraded badge. TIER_3 = prominent halt banner ("Human review required — no automated action taken"). The UI must never imply an action was taken on a TIER_3.
5. **TypeScript strict.** The frontend has its own `tsconfig.json` with `strict: true` and the same strictness posture as the root project (`noUncheckedIndexedAccess`, `noImplicitAny`, `noUnusedLocals`). Must pass `tsc --noEmit` before commit.
6. **Additive only.** Do not modify any existing backend file under `lib/**`, `modules/**`, or `scripts/**` except to import types from them. The data contracts below are the source of truth; mirror them, do not redefine them loosely.
7. **Preview-host safe.** Dev server must be reachable through the VM preview URL (configure `allowedHosts` / `allowedDevOrigins` — see §7).

---

2. STACK

- **Framework:** Next.js 14 (App Router) + React 18 + TypeScript.
- Rationale: the data lives as local JSON files. Next.js **Server Components / Route Handlers** can read `data/**` from the filesystem at request time with `node:fs`, keeping all file access server-side (Rule 2). No client-side fetch of secrets, no API keys shipped to the browser.
- **Styling:** Tailwind CSS (utility-first, no runtime CSS-in-JS). Dark, dense “operator console” aesthetic.
- **Charts:** `recharts` (React-native, tree-shakeable). Optional — tables are acceptable if a chart adds no clarity.
- **No state library.** Server Components fetch; a thin client layer handles only view toggles (node filter, tier filter). `useState` / `useReducer` only.
- **Package manager:** npm. `/frontend` gets its **own** `package.json` (isolated from the backend runtime deps).

---

3. DIRECTORY LAYOUT TO CREATE

```
frontend/
  package.json                 # isolated frontend deps + scripts
  tsconfig.json                # strict; path alias @backend/* -> ../
  next.config.mjs              # allowedDevOrigins for preview host
  tailwind.config.ts
  postcss.config.mjs
  .eslintrc.json
  app/
    layout.tsx                 # shell: sidebar nav + global alert banner slot
    globals.css
    page.tsx                   # Overview (weekly dossier summary)
    dossier/page.tsx           # Forward Intelligence — full weekly dossier
    margins/page.tsx           # Margin & Dead-Stock Scanner view
    competitors/page.tsx       # Competitor Radar view
    api/
      dossier/route.ts         # GET latest weekly-dossier JSON
      margins/route.ts         # GET latest margin-scan JSON
      competitors/route.ts     # GET latest competitor-sweep JSON
  lib/
    data-loader.ts             # server-only: locate + parse latest artifact
    types.ts                   # re-export/import backend types (see §4)
    format.ts                  # currency/pct/date/days helpers
    empty-state.ts             # canonical "no data" detection
  components/
    AppShell.tsx
    NavSidebar.tsx
    AlertBanner.tsx            # TIER_2 badge / TIER_3 halt banner
    StatCard.tsx
    EmptyState.tsx
    DataStamp.tsx              # shows generatedAt + source file name
    tables/VendorTable.tsx
    tables/ReorderTable.tsx
    tables/MarginTable.tsx
    tables/CompetitorTable.tsx
```

---

4. DATA CONTRACTS (SOURCE OF TRUTH — MIRROR EXACTLY)

The loaders MUST type their output against the backend interfaces. Import them via the `@backend/*` path alias where possible; otherwise mirror them **verbatim** in `frontend/lib/types.ts` with a comment pointing at the origin file. Do not loosen field names or optionality.

4.1 Forward Intelligence — Weekly Dossier

Origin: `modules/forward-intelligence/dossier-synthesizer.ts` → `WeeklyDossier`
Artifact glob: `data/forward-intel/weekly-dossier-*.json`

```
interface WeeklyDossier {
  generatedAt: string;
  inventoryHealth: {
    skusAnalyzed: number;
    skusSkippedNoCost: number;
    marginWarningCount: number;
    marginCriticalCount: number;
    deadStockCount: number;
  };
  reorderWatch: {
    totalReorder: number;
    totalOverstock: number;
    topReorder: ReorderAlert[];
    topOverstock: ReorderAlert[];
  };
  vendorRankings: {
    totalVendors: number;
    top3: VendorScorecard[];
    bottom3: VendorScorecard[];
  };
  nodeComparison: Array<{
    nodeId: string;
    totalSKUs: number;
    reorderCount: number;
    overstockCount: number;
  }>;
}
```

`ReorderAlert` (origin `demand-forecaster.ts`): `productId, name, vendorName, quantityAvailable, daysOnHand, unitCost, triggerType: 'REORDER'|'OVERSTOCK'`.
`VendorScorecard` (origin `vendor-scorecard.ts`): `vendorId, vendorName, skuCount, avgGrossMarginPct, avgDaysOnHand, deadStockCount, compositeScore, rank`.

4.2 Margin & Dead-Stock Scanner

Origin: `modules/margin-scanner/scanner.ts` → `MarginScanResult`
Artifact glob: `data/margin-scans/margin-scan-*.json`

```ts
interface MarginScanResult {
  scanId: string;
  startedAt: string;
  finishedAt: string;
  skusAnalyzed: number;
  skusSkippedNoCost: number;
  marginWarnings: MarginFlag[];   // label MARGIN_WARNING, tier TIER_1|TIER_2
  marginCritical: MarginFlag[];   // label MARGIN_CRITICAL
  deadStockCandidates: DeadStockFlag[];
}
```

`MarginFlag`: `node, productName, category?, quantityAvailable, unitCost, recPrice, grossMarginPct, label, alertTier`.
`DeadStockFlag`: `node, productName, category?, quantityAvailable, lastModifiedDateUTC: string|null, daysSinceLastModified: number|null, label: 'DEAD_STOCK_CANDIDATE', alertTier: TIER_2`.

> **KNOWN LIMITATION to surface in UI:** `daysSinceLastModified` is derived from `lastModifiedDateUTC` (catalog modification), NOT true last-sale. Render a small info tooltip on the dead-stock column stating this proxy caveat. Do not relabel it “days since sold”.

4.3 Competitor Radar

Origin: `modules/competitor-radar/scraper.ts` → `SweepResult`
Artifact glob: `data/competitor-sweeps/*.json`

```ts
interface SweepResult {
  sweepId: string;
  startedAt: string;
  finishedAt: string;
  targetCount: number;
  successCount: number;
  failureCount: number;
  snapshots: Array<{
    target: string;
    dispensarySlug: string;
    fetchedAt: string;
    ok: boolean;
    products: Array<{ name: string; category?: string; price?: number }>;
    note?: string;
  }>;
}
```

---

5. DATA LOADER SPEC (`frontend/lib/data-loader.ts`)

- Mark the module server-only (`import 'server-only';`). It uses `node:fs/promises` + `node:path`.
- Resolve the project root as `path.resolve(process.cwd(), '..')` when Next runs from `/frontend` (make this configurable via `DATA_ROOT` env, default `..`).
- `loadLatestArtifact<T>(dir: string, prefix: string): Promise<LoadResult<T>>`:
- List `data/<dir>`, filter files matching `<prefix>*.json`, pick the **newest by embedded timestamp in filename, then mtime tiebreak**.
- Parse JSON. On success return `{ status: 'ok', data, sourceFile, loadedAt }`.
- If the directory/file is missing → `{ status: 'missing' }`.

- If JSON is malformed → `{ status: 'error', message }`. **Never throw to the page**; the page renders the corresponding empty/error state.
- Provide typed wrappers: `loadLatestDossier()`, `loadLatestMarginScan()`, `loadLatestSweep()`.
- `LoadResult<T>` is a discriminated union on `status` (`'ok'|'missing'|'error'`). Pages switch on it — this is how Rule 3 (no fabrication) is enforced structurally.

Empty-data detection (`empty-state.ts`): even a successfully-parsed dossier can be semantically empty (the current verified artifact has `skusAnalyzed:0`, `totalVendors:0`, empty `nodeComparison`). Provide `isDossierEmpty()`, `isMarginScanEmpty()`, `isSweepEmpty()` helpers; when true, the page shows an `EmptyState` with the reason "Upstream catalog unavailable during this run" and still shows the `DataStamp` (generatedAt + file), so the operator sees the run happened but returned nothing.

---

6. PAGE / VIEW SPEC

6.1 `/` — Overview

- Top row of `StatCard`s from `weeklyDossier.inventoryHealth`: SKUs Analyzed, Skipped (no cost), Margin Warnings, Margin Critical, Dead Stock.
- Reorder Watch mini-summary: totalReorder vs totalOverstock.
- Vendor snapshot: top vendor (`top3[0]`) if present.
- `nodeComparison` two-node table (5th Ave vs 9th Ave).
- Global `AlertBanner` reads run health: if the dossier is empty → degraded banner; if a TIER_3 marker is present in a companion run log → halt banner.
- `DataStamp` always visible.

6.2 `/dossier` — Forward Intelligence (full)

- Full Reorder Watch: `ReorderTable` for `topReorder` and `topOverstock` (columns: name, vendor, qty, daysOnHand, unitCost, trigger).
- Full Vendor Rankings: `VendorTable` for top3 + bottom3 (columns: rank, vendor, composite, margin%, avgDOH, SKUs, dead). If `totalVendors <= 3`, note that bottom3 is intentionally empty (mirror backend behavior — no duplicate listing).
- Node comparison chart (recharts bar) or table fallback.

6.3 `/margins` — Margin & Dead-Stock

- Two `MarginTable`s: Warnings and Critical (columns: node, product, category, qty, unitCost, recPrice, grossMargin%, tier badge).
- Dead-Stock table with the KNOWN LIMITATION tooltip on the days column.
- Header stats: skusAnalyzed / skusSkippedNoCost / counts per bucket.

6.4 `/competitors` — Competitor Radar

- Sweep header: targetCount / successCount / failureCount + timestamps.
- Per-target `CompetitorTable` cards keyed by `dispensarySlug`; failed targets (`ok:false`) render a degraded card showing `note`, not an empty product grid.

Alert rendering rules (`AlertBanner`, `StatCard` badges):
- TIER_1 → neutral/info.
- TIER_2 → amber "Degraded" badge.
- TIER_3 → red full-width halt banner: "HUMAN REVIEW REQUIRED — operation halted, no automated action taken."

7. CONFIG REQUIREMENTS

- `frontend/next.config.mjs`: set `allowedDevOrigins: ['*']` (or the exact preview host) so the dev server is reachable through the VM preview URL. Bind dev server to a **non-reserved** port (default 3000; never 1000/2200).
- `frontend/tsconfig.json`: `strict: true`, `noUncheckedIndexedAccess: true`, `noUnusedLocals: true`, `paths: { "@backend/*": ["../*"] }`, `"@/*": ["./*"]`.
- `frontend/package.json` scripts:
- `"dev": "next dev -p 3000"`
- `"build": "next build"`
- `"start": "next start -p 3000"`
- `"typecheck": "tsc --noEmit"`
- `"lint": "next lint"`
- Root `package.json` (additive): add `"frontend:dev": "npm --prefix frontend run dev"` and `"frontend:build": "npm --prefix frontend run build"` for convenience. Do not disturb existing scripts.
- `.gitignore` (additive): ignore `frontend/.next/`, `frontend/node_modules/`.

8. BUILD SEQUENCE (ORDERED)

1. Scaffold `/frontend` skeleton (config files, empty app shell). Confirm `npm --prefix frontend install` succeeds.
2. Implement `lib/types.ts` (mirror §4), `lib/data-loader.ts`, `lib/empty-state.ts`, `lib/format.ts`.
3. Implement `api/*/route.ts` handlers delegating to the loaders (return `LoadResult` JSON with correct HTTP status: 200 ok, 200 with `status:'missing'` body — never 500 for missing data).
4. Build shared components (`AppShell`, `NavSidebar`, `AlertBanner`, `StatCard`, `EmptyState`, `DataStamp`).
5. Build pages `/`, `/dossier`, `/margins`, `/competitors` as Server Components consuming the loaders directly.
6. Wire the tables.
7. Verify against the **real** current artifact (the empty verified dossier) — the empty-state path MUST render cleanly, not crash.
8. Run acceptance gates (§9). Commit.

9. ACCEPTANCE GATES (ALL MUST PASS BEFORE COMMIT)

- [ ] `npm --prefix frontend install` completes with no errors.
- [ ] `npm --prefix frontend run typecheck` (`tsc --noEmit`) exits **0** in strict mode.
- [ ] `npm --prefix frontend run build` (`next build`) succeeds.
- [ ] Dev server loads through the **VM preview URL** (not just localhost) with HTTP 200 — no “host not allowed” 403.

- [ ] With the current verified artifact (`weekly-dossier-1787270627557.json`, all-zero), `/` and `/dossier` render the **EmptyState + DataStamp** — no fabricated numbers, no crash.
- [ ] `/margins` and `/competitors` render a clean "no artifact yet" state (those dirs have no data file yet) without throwing.
- [ ] No secret, API key, or `DUTCHIE_*` / AWS value appears anywhere under `/frontend` or in the client bundle (grep clean).
- [ ] No file under `lib/**`, `modules/**`, `scripts/**` modified except type imports.
- [ ] TIER_3 halt banner and TIER_2 degraded badge components exist and render from state.

Commit message: `feat: frontend operator console — scaffold /frontend, wire read-only data loaders`
Surface all new files in full after commit.

---

10. OUT OF SCOPE (DO NOT BUILD)

- Any write-back, price push, reorder submission, or "apply recommendation" button.
- Any live Dutchie/AWS call from the frontend.
- Auth/login, multi-tenant, or user management (single-operator local console).
- Notifications of any kind (email/SMS/webhook/push).
- Editing or re-computing backend numbers client-side.

Appendix A — Rationale notes for the executing agent

- **Why Server Components read files directly:** keeps filesystem + any future secret-bearing logic on the server, satisfying Rule 2 structurally rather than by convention.
- **Why a discriminated `LoadResult`:** forces every page to handle `missing` / `error` explicitly, making Rule 3 (no fabrication) a compile-time obligation, not a guideline.
- **Why mirror backend types:** the artifacts are the contract. If the backend interfaces change, `tsc` against `@backend/*` imports will flag drift immediately.